

PROJECT REPORT

EXPERIMENT – 08

Project title:

Wireless Environmental Parameter Monitoring System for Smart Industries

Submitted by:

Arundepak S -24MER004
Deepak P -24MER006
Dharanidharan R S -24MER007
Dinesh k -24MER009
Vishnuvarthan P -24MER063

1. Project Overview

This project presents the design and development of a Wireless Environmental Parameter Monitoring System intended for smart industrial environments. The system continuously measures key environmental parameters — temperature, humidity, and air quality — using low-cost sensors interfaced with an ESP8266 NodeMCU microcontroller. Sensor readings are displayed locally on an LCD, an audible buzzer alerts personnel to hazardous conditions, and all data is transmitted wirelessly over Wi-Fi to the ThingSpeak IoT cloud platform for remote, real-time monitoring and historical trend analysis.

The solution demonstrates a complete embedded-to-cloud IoT pipeline: sensing, local display/alerting, wireless connectivity, and cloud data logging/visualization, making it directly applicable to Industry 4.0 environmental safety and asset-monitoring use cases.

2. Problem Statement

Industrial environments are often subject to fluctuations in temperature, humidity, and air quality due to machinery operation, chemical processes, and poor ventilation. Uncontrolled variation in these parameters can:

- Damage sensitive equipment and reduce product quality
- Create unsafe or unhealthy working conditions for personnel
- Go undetected until the situation becomes critical, since manual/on-site checks are infrequent and not real-time
- Lack centralized, remote visibility for supervisors and facility managers

There is therefore a need for an automated, low-cost, wireless system that can continuously monitor these parameters, provide immediate on-site alerts, and make the data remotely accessible for proactive decision-making.

3. Proposed Solution

The proposed system uses an ESP8266 NodeMCU as the central controller, interfaced with a DHT11 sensor (temperature & humidity) and an MQ135 gas sensor (air quality). Key features of the solution:

- Real-time sensing of temperature, humidity, and air quality (gas/smoke concentration)
- On-site 16x2 I2C LCD display for immediate readability by plant personnel
- Automatic buzzer alert when air quality is poor or temperature exceeds a safe threshold ($\geq 40^{\circ}\text{C}$)
- Wireless data upload to the ThingSpeak cloud every 20 seconds via Wi-Fi (HTTP GET)
- Remote dashboards on ThingSpeak allow trend visualization of all four monitored fields from anywhere
- Modular, low-cost hardware suitable for scaling across multiple industrial zones

4. System Architecture

The system follows a three-layer IoT architecture:

- Perception Layer: DHT11 (temperature & humidity) and MQ135 (air quality) sensors capture raw environmental data.
- Processing / Edge Layer: The ESP8266 NodeMCU reads sensor data, evaluates alert conditions, drives the LCD and buzzer, and formats data for transmission.
- Network & Cloud Layer: Data is sent over Wi-Fi using HTTP GET requests to the ThingSpeak REST API, where it is stored and visualized on live charts (Field 1–4).

Data flow: Sensors → ESP8266 (processing & alert logic) → LCD/Buzzer (local feedback) → Wi-Fi → ThingSpeak Cloud (remote monitoring).

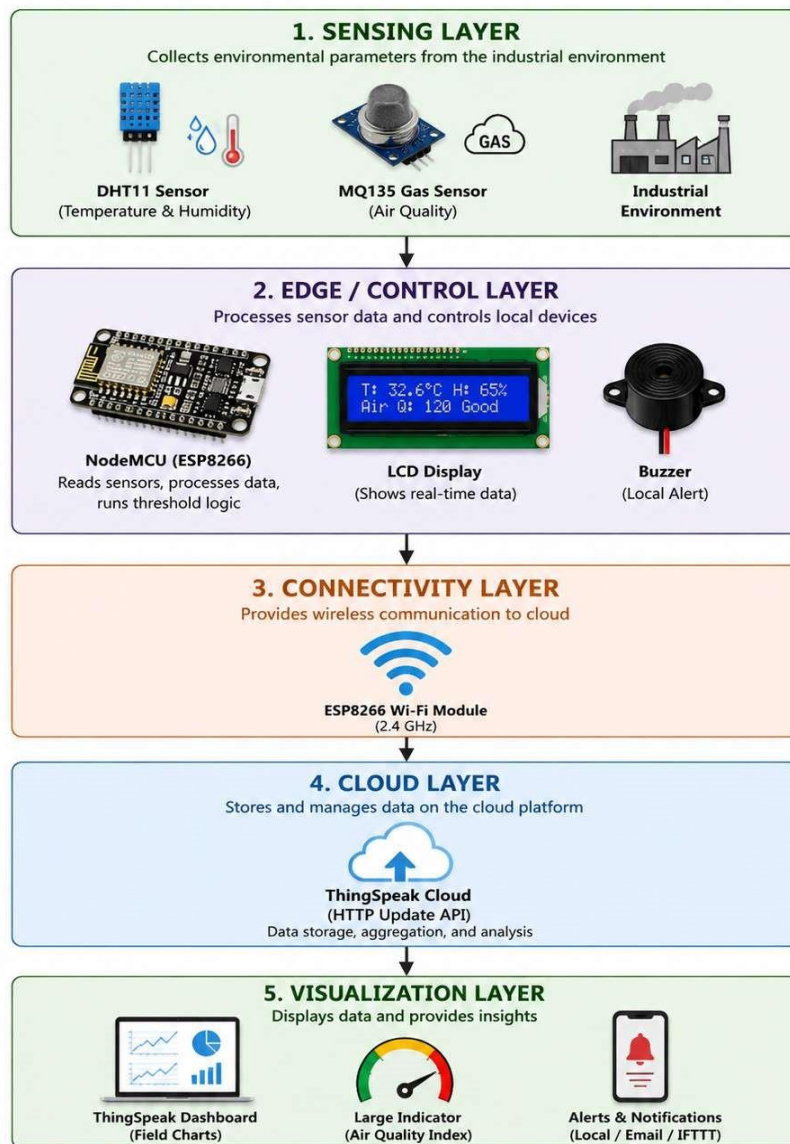


Figure 1: system architecture

5. Circuit Table

Component	Pin	ESP8266 NodeMCU Pin	GPIO
DHT11 Sensor	Data	D4	GPIO2
MQ135 Gas Sensor	Analog Out	A0	A0
Buzzer	Signal (+)	D5	GPIO14
I2C LCD (16x2)	SDA	D2	GPIO4
I2C LCD (16x2)	SCL	D1	GPIO5
All Modules	VCC / GND	3V3 / G	—

6. Circuit Design

The DHT11, MQ135, buzzer, and I2C LCD are wired to the ESP8266 NodeMCU as per the circuit table above, with common power (3.3V) and ground rails distributed via the breadboard. The DHT11 data line connects to GPIO2 (D4), the MQ135 analog output connects to the single analog pin A0, the buzzer is driven from GPIO14 (D5), and the LCD communicates over I2C using GPIO4 (SDA) and GPIO5 (SCL) — a bit-banged I2C bus configured explicitly in software via `Wire.begin(4, 5)`.

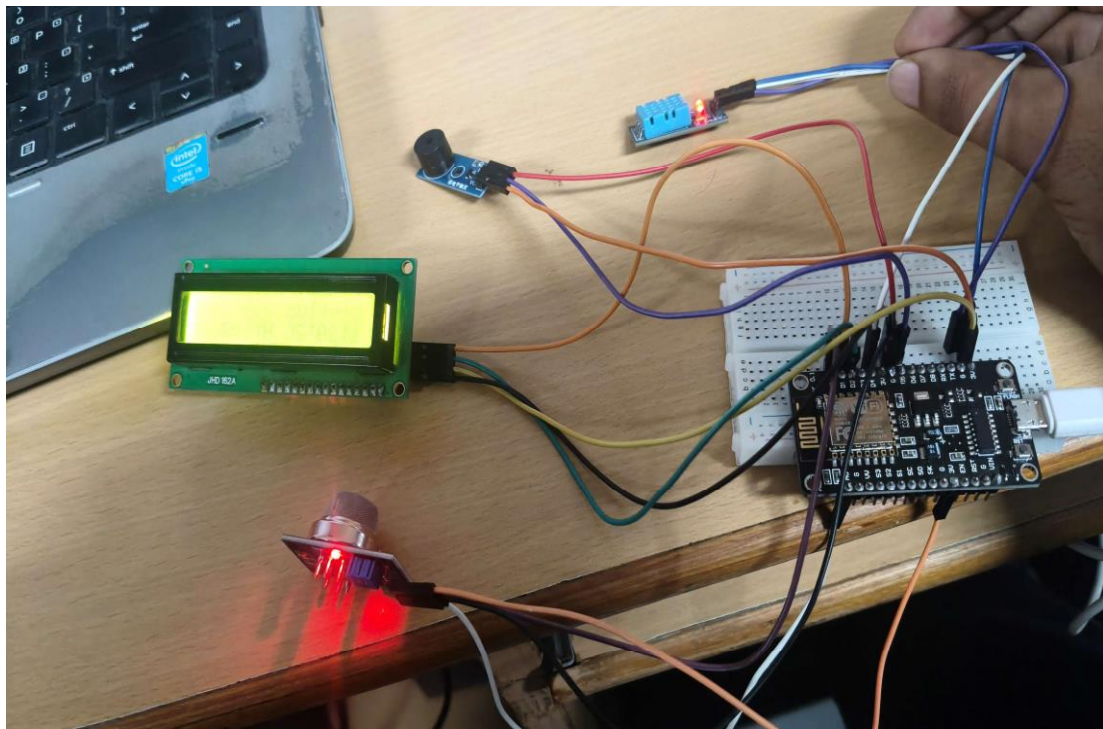


Figure 2: Assembled hardware prototype on breadboard

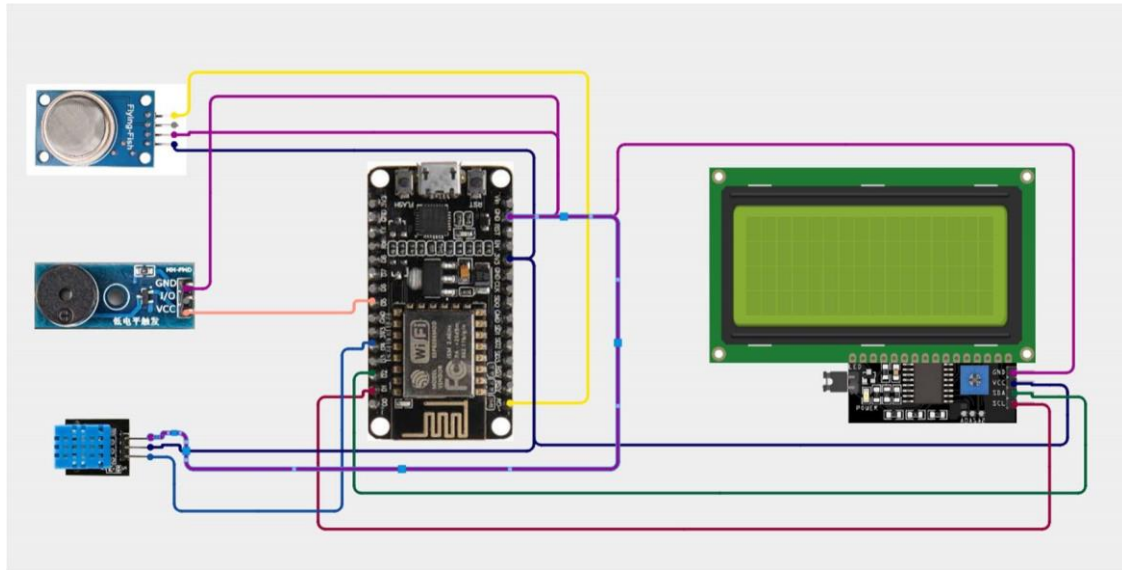


Figure 3: Circuit wiring diagram (Fritzing) — ESP8266 NodeMCU with DHT11, MQ135, buzzer, and I2C LCD

7. Algorithm

Step-by-step operating logic implemented in firmware:

- 1. Initialize Serial, DHT11 sensor, buzzer pin, I2C bus and LCD; display a startup message.
- 2. Connect to the configured Wi-Fi network and display connection status on the LCD.
- 3. In the main loop, read temperature and humidity from DHT11 and the analog gas concentration from MQ135.
- 4. Validate DHT11 readings; if invalid (NaN), display an error on the LCD and skip this cycle.
- 5. Classify air quality as GOOD (<300), MODERATE (300–599), or POOR (≥ 600) based on the MQ135 reading.
- 6. Evaluate the alert condition: trigger the buzzer if air quality ≥ 600 OR temperature $\geq 40^{\circ}\text{C}$; otherwise keep the buzzer off.
- 7. Print all readings to the Serial Monitor and update the LCD with temperature, humidity, and air quality.
- 8. If Wi-Fi is connected, build a ThingSpeak REST URL with the four field values and send it via HTTP GET; log the response/entry ID. If disconnected, attempt to reconnect.
- 9. Wait 20 seconds (ThingSpeak's minimum update interval) and repeat from Step 3.

8. Coding

The complete Arduino/ESP8266 firmware (C++) implementing the algorithm above is listed below.

```
#include <ESP8266WiFi.h>

#include <ESP8266HTTPClient.h>


#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <DHT.h>


const char* WIFI_SSID = "test" const char*
WIFI_PASSWORD = "test1234";


const char* THINGSPEAK_API_KEY = "JPVUJWJT6OODYLBR";


#define DHTPIN 2

#define DHTTYPE DHT11

#define MQ135_PIN A0

#define BUZZER_PIN 14


DHT dht(DHTPIN, DHTTYPE);

LiquidCrystal_I2C lcd(0x27, 16, 2);


void setup() {
    Serial.begin(115200);

    dht.begin();

    pinMode(BUZZER_PIN, OUTPUT);  digitalWrite(BUZZER_PIN,
LOW);
```

```
Wire.begin(4, 5);

lcd.init(); lcd.backlight();
lcd.clear(); lcd.setCursor(0,
0);
lcd.print("Environmental");
lcd.setCursor(0, 1);
lcd.print("Monitoring");
delay(2000);

lcd.clear(); lcd.setCursor(0,
0); lcd.print("Connecting
WiFi");

WiFi.begin(WIFI_SSID, WIFI_PASSWORD); while
(WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
} lcd.clear();
lcd.setCursor(0, 0);
lcd.print("WiFi Connected");
lcd.setCursor(0, 1);
lcd.print("IP Ready");
delay(2000); lcd.clear();
}

void loop() { float temperature =
dht.readTemperature(); float humidity =
dht.readHumidity(); int airQuality =
analogRead(MQ135_PIN);
```

```
    if (isnan(temperature) || isnan(humidity)) {  
    lcd.clear();    lcd.setCursor(0, 0);  
    lcd.print("DHT11 ERROR");  
    digitalWrite(BUZZER_PIN, LOW);  
    delay(2000);  
    return;  
    }
```

```
    String airStatus; if (airQuality < 300) airStatus =  
    "GOOD"; else if (airQuality < 600) airStatus =  
    "MODERATE"; else airStatus = "POOR";
```

```
    int alertStatus = 0; if (airQuality >= 600 ||  
    temperature >= 40) {  
    digitalWrite(BUZZER_PIN, HIGH);  
    alertStatus = 1;  
    } else {  
    digitalWrite(BUZZER_PIN, LOW);  
    alertStatus = 0;  
    }
```

```
    lcd.clear();  
    lcd.setCursor(0, 0);  
    lcd.print("T:");  
    lcd.print(temperature, 1);  
    lcd.print("C ");  
    lcd.print("H:");
```



```
lcd.print(humidity, 0);  
lcd.print("%");  
lcd.setCursor(0, 1);  
lcd.print("Air:");  
lcd.print(airQuality);  
delay(2000);  
  
if (WiFi.status() == WL_CONNECTED) {  
    WiFiClient client;  
    HTTPClient http;  
    String serverURL = "http://api.thingspeak.com/update?api_key=";  
serverURL += THINGSPEAK_API_KEY;  
    serverURL += "&field1=" + String(temperature);  
serverURL += "&field2=" + String(humidity);  
serverURL += "&field3=" + String(airQuality);  
serverURL += "&field4=" + String(alertStatus);  
    http.begin(client, serverURL);    int  
    httpResponseCode = http.GET();    http.end();  
    } else {  
        WiFi.begin(WIFI_SSID, WIFI_PASSWORD);  
    }  
    delay(20000);  
}
```

9. System Working

On power-up, the ESP8266 initializes all peripherals and connects to the configured Wi-Fi network, confirming this on the LCD. During normal operation, the DHT11 and MQ135 sensors are sampled every cycle. The current temperature, humidity, and air-quality index are shown on the LCD for immediate on-site visibility. If the air quality crosses the POOR threshold or temperature reaches 40°C, the buzzer sounds automatically as a local safety alert. Simultaneously, every reading is pushed to the ThingSpeak channel over Wi-Fi via an HTTP GET request, updating four separate live charts (Temperature, Humidity, Air Quality, Alert Status) that can be viewed remotely from any device. This closes the loop between physical sensing, local alerting, and remote/cloud-based monitoring.

10. Bill of Materials

S.No	Component	Quantity
1	ESP8266 NodeMCU	1
2	DHT11 / DHT22 Sensor	1
3	MQ135 Air Quality Sensor	1
4	I2C 16x2 LCD Display	1
5	Buzzer	1
6	Breadboard	1
7	Jumper Wires	1 Set

11. Prototype Implementation

The firmware was developed and uploaded to the ESP8266 NodeMCU using Arduino IDE 2.3.10 (board: Generic ESP8266 Module) with the Adafruit DHT sensor library. After flashing, the Serial Monitor was used to verify sensor readings, air-quality status classification, alert flag, and successful HTTP responses (Response 200) from ThingSpeak with incrementing entry IDs, confirming reliable end-to-end operation.

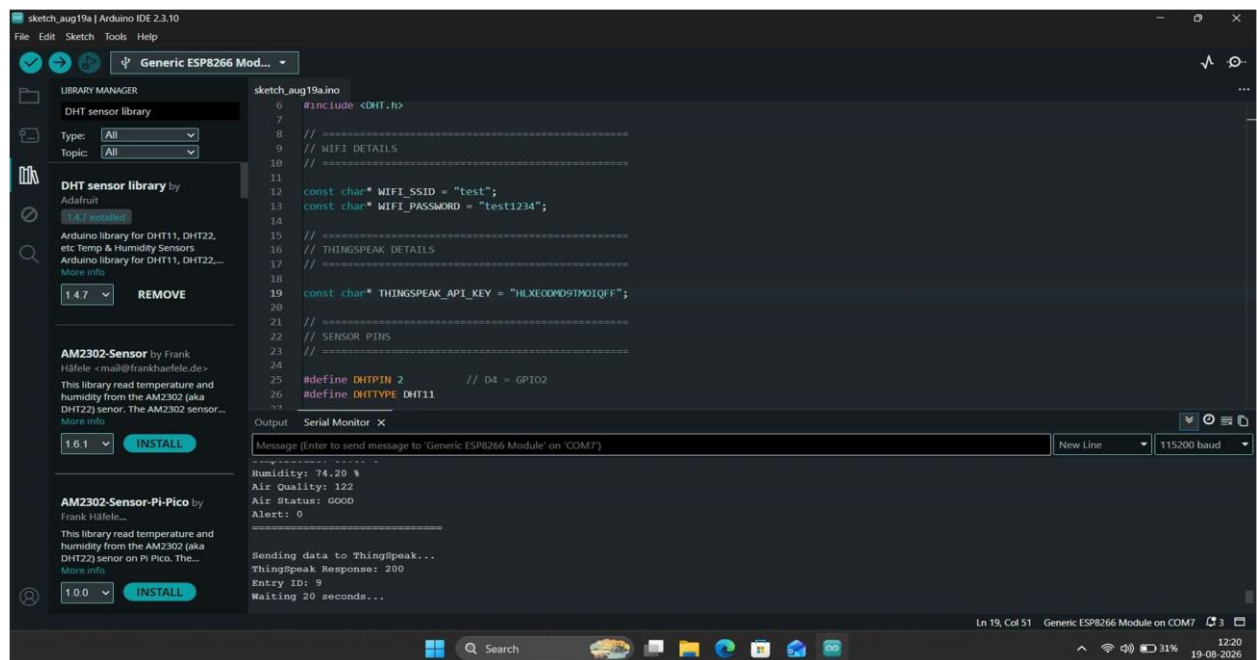


Figure 4: ThingSpeak dashboard after 5 entries

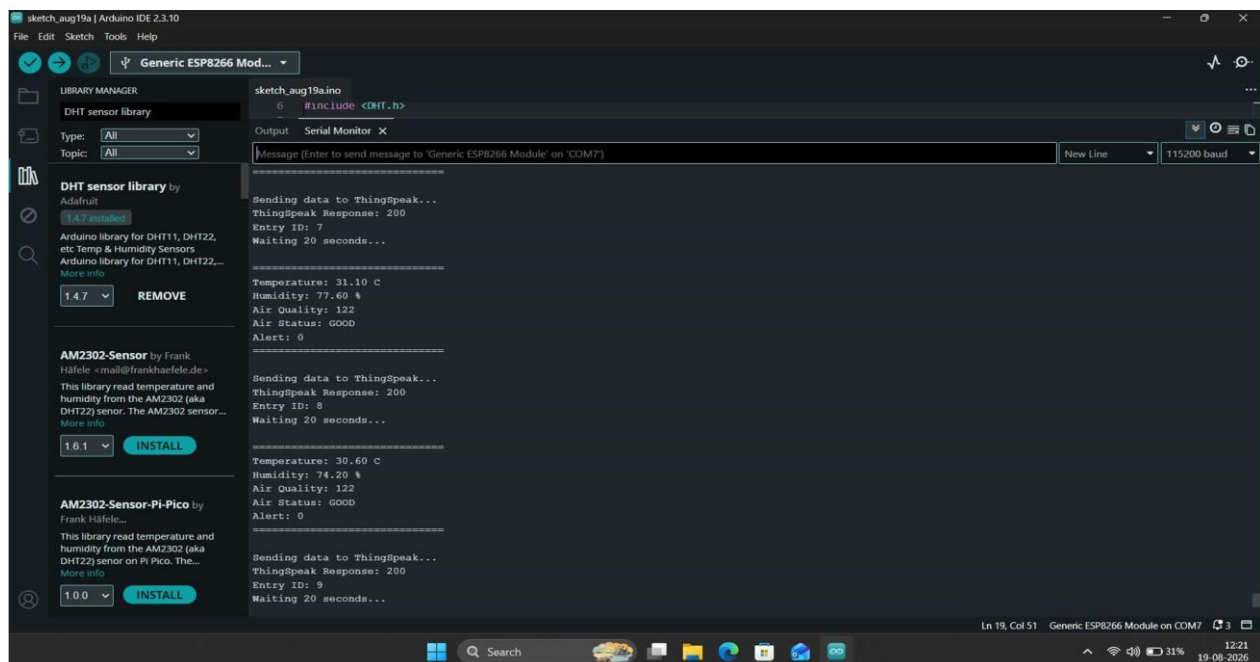


Figure 5: ThingSpeak dashboard after 7 entries, showing continued upward trend

12. Results & Performance Analysis

The system was tested continuously for several update cycles (20-second interval) with data successfully logged to a ThingSpeak channel titled "ex 11 air quality". The four ThingSpeak charts below confirm correct end-to-end functioning:

- Field 1 — Temperature (°C): remained stable around 29.5–30°C initially, rising to ~32°C over the test window.
- Field 2 — Humidity (%): tracked between 68% and 95%, rising in correlation with temperature.
- Field 3 — Air Quality (raw ADC): increased from ~112 to ~122, staying within the GOOD range (<300) throughout testing.

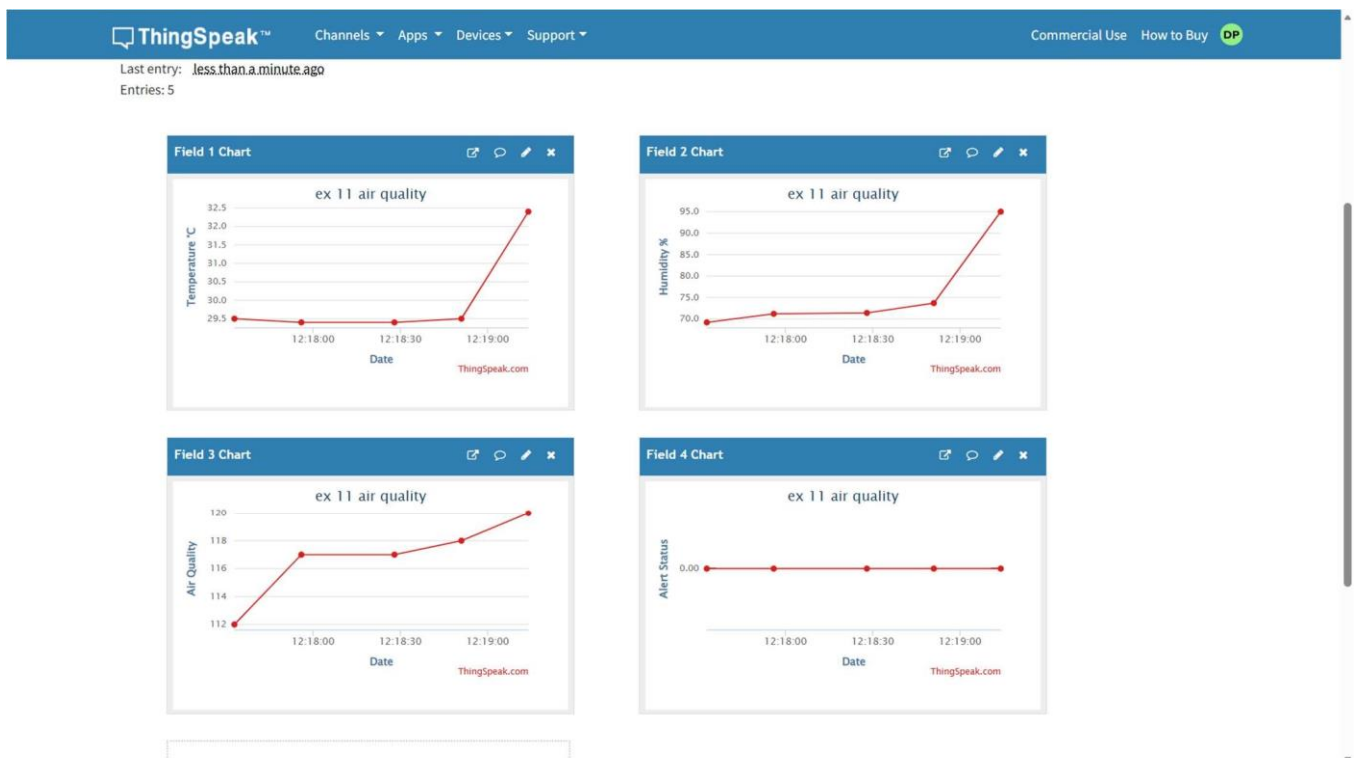


Figure 6: Arduino IDE — sketch, board selection, and live Serial Monitor output

Last entry: less than a minute ago
Entries: 7

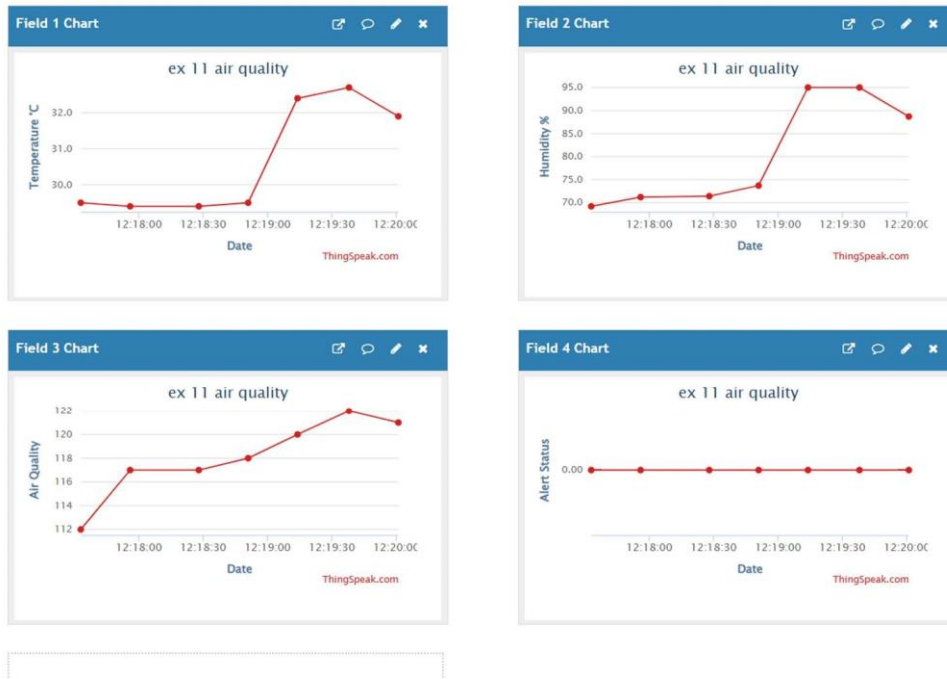


Figure 7: Serial Monitor showing successful ThingSpeak uploads with entry IDs

- Field 4 — Alert Status: remained at 0 throughout, correctly indicating no hazardous condition was triggered, consistent with the observed readings staying below the alert thresholds (air quality < 600, temperature < 40°C).

Performance Summary: All HTTP requests to ThingSpeak returned response code 200, confirming 100% successful data delivery during testing with no dropped uploads. The alert logic correctly withheld the buzzer trigger since neither threshold was crossed. The 20-second update interval respects ThingSpeak's free-tier rate limit while providing near real-time visibility. The system demonstrates a functional, low-cost, and scalable solution for wireless environmental monitoring in smart industrial settings.